INF672 · MACHINE LEARNING · UNIDAD 3

Práctica 3

Regresión polinomial: predecir el rendimiento de combustible de un automóvil

En esta práctica vas a comprobar que un modelo lineal puede ajustar una **curva**. Entrenarás un modelo de **regresión polinomial** que prediga el rendimiento de combustible de un automóvil a partir de su potencia, y lo compararás con una regresión lineal simple. Como en la Práctica 2, los pasos forman **un solo flujo de trabajo**: cada paso usa lo que obtuviste en el anterior. Lee el **contexto** de cada paso, sigue las instrucciones en orden y responde las preguntas del bloque **Para entregar**.

Esta práctica es un **trabajo individual**. Las respuestas escritas deben apoyarse en los resultados que tú obtengas al ejecutar tu código: no basta con mostrar números o gráficos, hay que interpretarlos en el contexto del problema.

Para entregar tu trabajo desarrolla un notebook de Jupyter que incluya tanto el código ejecutado como tus respuestas escritas en celdas de texto, organizado con un título por cada paso. Súbelo a tu repositorio de GitHub junto con el archivo de datos y envía el enlace a la plataforma virtual en el apartado de Práctica 3.

EL PROBLEMA

¿Cuántas millas recorre un auto por galón?

Una organización de defensa del consumidor quiere estimar el rendimiento de combustible de un automóvil conociendo solo su potencia, un dato que aparece en cualquier ficha técnica. Con esa estimación podría advertir a los compradores sobre el consumo esperado de un vehículo antes de adquirirlo.

La intuición dice que a mayor potencia, menor rendimiento. Pero la relación no tiene por qué ser una línea recta: tu tarea es averiguar qué forma tiene, construir un modelo que la represente bien y decidir qué tan complejo debe ser ese modelo.

La **regresión polinomial** permite ajustar curvas sin abandonar la regresión lineal: se agregan columnas con potencias de la variable de entrada y el modelo sigue siendo lineal en sus coeficientes:

$$\hat{y} = \beta_0 + \beta_1 \cdot x + \beta_2 \cdot x^2 + \ldots + \beta d \cdot x^d$$

donde *d* es el **grado** del polinomio. Elegir bien ese grado es una de las decisiones centrales de esta práctica.

EL DATASET

Archivo mpg.csv

El archivo `mpg.csv` contiene **398 filas** (una por automóvil) y **9 columnas**. Es el conjunto de datos público *Auto MPG*, con modelos de automóviles fabricados entre 1970 y 1982. La variable a predecir es `mpg`; en esta práctica usarás como predictor principal `horsepower`.

Variable	Tipo	Descripción
`mpg`	Numérica (objetivo)	**Rendimiento de combustible**: millas recorridas por galón. A mayor valor, menor consumo.
`cylinders`	Numérica discreta	Número de cilindros del motor.
`displacement`	Numérica	Cilindrada del motor, en pulgadas cúbicas.
`horsepower`	Numérica	Potencia del motor, en caballos de fuerza (hp).
`weight`	Numérica	Peso del vehículo, en libras.
`acceleration`	Numérica	Tiempo, en segundos, para acelerar de 0 a 60 millas por hora.
`model_year`	Numérica discreta	Año del modelo, en dos dígitos (70 = 1970).
`origin`	Categórica	Región de fabricación: `usa`, `europe` o `japan`.
`name`	Texto	Marca y modelo del automóvil.

Revisa la calidad de los datos antes de modelar: los modelos de scikit-learn no aceptan valores vacíos, y es tu responsabilidad detectarlos y decidir qué hacer con ellos.

PASO 01

Preparar el entorno de trabajo

Además de las librerías que ya usaste en la Práctica 2, necesitarás herramientas para transformar variables, encadenar pasos y validar modelos.

1. Crea un notebook nuevo y coloca el archivo `mpg.csv` en la misma carpeta.
2. Importa `pandas`, `numpy` y `matplotlib.pyplot`.
3. Desde scikit-learn importa: `train_test_split`, `cross_val_score` y `KFold` (módulo `model_selection`); `PolynomialFeatures` y `StandardScaler` (módulo `preprocessing`); `LinearRegression` (módulo `linear_model`); `make_pipeline` (módulo `pipeline`); y las métricas `mean_squared_error` y `r2_score` (módulo `metrics`).

> PARA ENTREGAR
> La celda de importaciones ejecutada sin errores y una tabla breve que indique, con tus palabras, para qué usarás cada función importada de scikit-learn.

PASO 02

Cargar y explorar los datos

Antes de elegir un modelo hay que conocer la tabla: cuántos registros hay, de qué tipo es cada columna y si falta información.

1. Carga el archivo con `pd.read_csv` y muestra sus dimensiones y primeras filas.
2. Revisa los tipos de datos con `df.info()` y cuenta los valores faltantes por columna con `df.isna().sum()`.
3. Obtén un resumen estadístico de las variables numéricas con `df.describe()`.

> PARA ENTREGAR
> Las salidas de las instrucciones. Responde: ¿cuántos automóviles y columnas tiene el dataset? ¿Hay valores faltantes? ¿En qué columna y cuántos? ¿Entre qué valores varían `mpg` y `horsepower`?

PASO 03

Elegir el predictor

Esta práctica usa una sola variable de entrada. Para justificar la elección, conviene medir qué tan relacionada está cada variable numérica con `mpg`.

1. Calcula la correlación de cada variable numérica con `mpg`. Puedes seleccionar las columnas numéricas con `df.select_dtypes("number")` y usar el método `.corr()`.
2. Ordena los resultados para identificar las relaciones más fuertes.

> PARA ENTREGAR
> La tabla de correlaciones ordenada. Responde: ¿qué variables tienen la relación más fuerte con `mpg`? ¿Qué significa el **signo** de esas correlaciones en términos del auto? Recuerda que la correlación de Pearson mide solo relaciones lineales: ¿por qué una variable con una relación claramente curva podría ser un buen candidato para la regresión polinomial aunque su correlación no sea la más alta?

PASO 04

Limpiar los datos

El modelo solo necesita dos columnas: `horsepower` como entrada y `mpg` como objetivo. Las filas con valores faltantes en esas columnas deben tratarse antes de entrenar.

1. Crea un nuevo DataFrame, llamado `datos`, que contenga solo las columnas `horsepower` y `mpg`.
2. Elimina las filas con valores faltantes usando `dropna()`.

3. Imprime cuántas filas quedaron.

> **PARA ENTREGAR**
> El número de filas antes y después de la limpieza. Responde: ¿qué porcentaje de los datos eliminaste? ¿Por qué en este caso es razonable eliminar esas filas en lugar de rellenarlas? ¿En qué situación preferirías imputar los valores?

PASO 05

Visualizar la relación

Antes de entrenar cualquier modelo conviene **mirar los datos**. Un diagrama de dispersión muestra si la relación entre dos variables es recta o curva, y eso decide qué tipo de modelo necesitas.

1. Dibuja un diagrama de dispersión con `horsepower` en el eje horizontal y `mpg` en el vertical.
2. Agrega título, etiquetas con unidades en los ejes y una cuadrícula.

> **PARA ENTREGAR**
> El gráfico y una descripción escrita de la forma de la relación. Responde: ¿el rendimiento disminuye al mismo ritmo en todo el rango de potencias? ¿Crees que una recta podrá representar bien estos datos? Justifica señalando zonas concretas del gráfico.

PASO 06

Dividir en entrenamiento y prueba

Como en la práctica anterior, el modelo se evaluará con datos que no usó para aprender. Esta vez, además, el conjunto de prueba deberá quedar **reservado** hasta el final: las decisiones sobre el modelo se tomarán solo con los datos de entrenamiento.

1. Define `X` con la columna `horsepower` usando doble corchete (`datos[["horsepower"]]`) e `y` con la columna `mpg`.
2. Divide los datos con `train_test_split`, reservando el 20 % para prueba y usando `random_state=42`.
3. Imprime cuántas filas quedaron en cada conjunto.

> **PARA ENTREGAR**
> El tamaño de cada conjunto. Responde: ¿por qué `X` se define con doble corchete y `y` con uno solo? ¿Qué forma (`.shape`) tiene cada uno y por qué scikit-learn exige esa diferencia?

PASO 07

Entrenar el modelo base: regresión lineal

El primer modelo es una regresión lineal simple, $\hat{y} = \beta_0 + \beta_1 \cdot x$. Sirve como **punto de comparación**: el modelo polinomial solo vale la pena si la supera. Usarás dos métricas: el **RMSE** (raíz del error cuadrático medio), que expresa el error en las mismas unidades que `mpg`, y el **R^2**.

1. Entrena un modelo `LinearRegression` con los datos de entrenamiento.
2. Predice el conjunto de prueba y calcula el RMSE (raíz cuadrada de `mean_squared_error`) y el R^2.

> PARA ENTREGAR
> Los valores de RMSE y R^2 del modelo lineal. Responde: ¿qué significa el RMSE en millas por galón? ¿Qué proporción de la variación del rendimiento explica este modelo?

PASO 08

Entrenar un modelo polinomial de grado 2

Un **pipeline** encadena varios pasos que se aplican siempre en el mismo orden, tanto al entrenar como al predecir. Para la regresión polinomial usarás tres: escalar la variable, generar sus potencias y ajustar una regresión lineal sobre las nuevas columnas.

1. Construye un pipeline con `make_pipeline(StandardScaler(), PolynomialFeatures(degree=2), LinearRegression())`, entrénalo y calcula sus métricas de prueba.
2. Accede a los pasos del pipeline con `named_steps` y muestra las columnas que `PolynomialFeatures` genera para los tres primeros autos de entrenamiento. Usa `get_feature_names_out` para ponerles nombre.
3. Dibuja en un mismo gráfico los puntos de entrenamiento y de prueba, la recta del modelo lineal y la curva del modelo polinomial. Para trazar líneas suaves genera 200 valores ordenados de potencia con `np.linspace`.

> PARA ENTREGAR
> Las métricas de ambos modelos, la tabla de columnas generadas y el gráfico comparativo. Responde: ¿qué columnas crea `PolynomialFeatures` y qué representa cada una? ¿Por qué se escala la variable antes de elevarla al cuadrado? ¿En qué zonas del gráfico la recta se equivoca más y cómo lo corrige la curva?

PASO 09

Elegir el grado con validación cruzada

Un grado muy bajo no captura la curva (**subajuste**); uno muy alto se adapta al ruido de los datos de entrenamiento (**sobreajuste**). Para elegir el grado sin usar el conjunto de prueba se aplica **validación cruzada** sobre los datos de entrenamiento.

1. Define una validación cruzada de 10 pliegues con `KFold(n_splits=10, shuffle=True, random_state=42)`.
2. Para cada grado de 1 a 10, construye el mismo pipeline del Paso 08 y calcula el RMSE medio con `cross_val_score` usando `scoring="neg_root_mean_squared_error"`. Ten en cuenta que scikit-learn devuelve este error con signo negativo.
3. Organiza los resultados en una tabla y dibuja el RMSE de validación cruzada en función del grado.
4. Identifica el grado con menor error y decide qué grado usarás en el modelo final.

> **PARA ENTREGAR**
> La tabla y el gráfico del error por grado, y el grado elegido. Responde: ¿cómo cambia el error al aumentar el grado? ¿El grado con menor error es necesariamente el mejor? Justifica tu elección aplicando el **principio de parsimonia** (entre modelos de rendimiento similar, se prefiere el más simple). ¿Por qué no se debe usar el conjunto de prueba para elegir el grado?

PASO 10

Evaluar el modelo final

Con el grado ya elegido, el modelo final se evalúa una sola vez con el conjunto de prueba. Además de las métricas, los **residuos** (valor real menos valor predicho) revelan si el modelo capturó la forma de los datos: si es adecuado, deben dispersarse al azar alrededor de cero, sin formar patrones.

1. Construye una tabla que compare el RMSE y el R^2 de prueba del modelo lineal y de tu modelo final.
2. Dibuja, uno al lado del otro, los residuos frente a los valores predichos de ambos modelos, con una línea horizontal en cero.
3. Para obtener una ecuación en unidades originales, reentrena tu modelo final **sin** `StandardScaler` y escribe la ecuación con sus coeficientes.

> **PARA ENTREGAR**
> La tabla comparativa, los gráficos de residuos y la ecuación del modelo. Responde: ¿cuánto mejoró el modelo final respecto al lineal? ¿Qué patrón observas en los residuos del modelo lineal y qué significa? ¿Desaparece en el modelo final? Interpreta el signo de cada coeficiente de la ecuación en relación con la forma de la curva.

PASO 11

Predecir y reconocer los límites del modelo

Un polinomio puede comportarse de forma extraña en los extremos del rango de los datos, y más aún fuera de él. Antes de usar el modelo hay que saber dónde son confiables sus predicciones.

1. Predice el rendimiento de autos con potencias de 60, 100, 150 y 200 hp usando tu modelo final.

2. Si tu modelo final es de grado 2, calcula el **vértice** de la parábola con $x_v = -\beta_1 / (2 \cdot \beta_2)$, usando los coeficientes de la ecuación en unidades originales. Si elegiste otro grado, grafica la curva en un rango amplio de potencias y localiza dónde cambia su tendencia.
3. Compara ese punto con el rango de potencias presente en los datos.

> PARA ENTREGAR
> La tabla de predicciones y el cálculo del vértice o el análisis gráfico. Responde: ¿las predicciones tienen sentido físico en todo el rango? ¿Qué predice el modelo más allá del vértice y por qué eso no es realista? ¿Qué es la **extrapolación** y por qué es riesgosa con modelos polinomiales?

PASO 12

Conclusiones

Cierra el trabajo con una síntesis escrita que responda al problema planteado.

1. Resume el procedimiento que seguiste y compara los resultados del modelo lineal y del modelo final.
2. Explica con tus palabras por qué la regresión polinomial sigue siendo una regresión lineal.

> PARA ENTREGAR
> Una conclusión de media página que responda: ¿qué forma tiene la relación entre potencia y rendimiento? ¿Qué grado recomendarías y por qué? ¿Qué limitaciones tiene el modelo y en qué rango de potencias confiarías en sus predicciones?

RETO ADICIONAL (OPCIONAL)

Ir más allá de una variable

Si terminaste los pasos anteriores, extiende el análisis para comprobar si otros predictores mejoran el modelo.

1. Repite el procedimiento de los Pasos 05 a 10 usando `weight` o `displacement` como predictor, y compara el grado elegido y las métricas con las de `horsepower`.
2. Entrena un modelo polinomial con varias variables a la vez (por ejemplo `horsepower`, `weight` y `model_year`) y compáralo con tu mejor modelo de una sola variable.

> PARA ENTREGAR
> La tabla comparativa de todos los modelos probados. Responde: ¿qué predictor individual funcionó mejor? ¿Cuánto mejora el modelo al combinar varias variables? ¿Cuántas columnas genera `PolynomialFeatures` con tres variables y grado 2, y qué representan las columnas nuevas?